MODULE

24

Authentication & security

Store passwords safely, issue tokens, check permissions, and stop the classic web attacks.

YOU WILL LEARN

Password hashing	`01_password_hashing.py`
JWT tokens	`02_jwt_tokens.py`
FastAPI login	`03_fastapi_login.py`
Security checklist	`04_web_security_checklist.py`
Interview Q&A	`INTERVIEW.md`

WHY SECURITY

One leak can end a company

THE PROBLEM

A database dump leaks. If passwords were stored as plain text or a fast hash, attackers log in as your users, and on every other site where they reused the password.

THE IDEA

Hash passwords slowly with a salt, give clients short-lived tokens, check permissions on every request, and validate all input.

STEP BY STEP AUTH BASICS

Authentication vs authorisation

	QUESTION	FAILURE	EXAMPLE
Authentication (authN)	**Who** are you?	**401** Unauthorized	password, token, OAuth login
Authorisation (authZ)	What may you **do**?	**403** Forbidden	roles, owner-only, admin-only

> Check authorisation on the **server** for every request. Hiding a button in the UI is not security.

COMMON MISTAKE PASSWORD HASHING

Storing passwords the wrong way

✗ WRONG

```
users[email] = password                    # plain text!
users[email] = hashlib.sha256(
    password.encode()).hexdigest()         # fast, unsalted
```

SHA-256 is built to be **fast**: a GPU tries billions per second. Without a salt, equal passwords have equal hashes (rainbow tables).

✓ RIGHT

```
salt = secrets.token_bytes(16)             # unique per user
key = hashlib.scrypt(password.encode(), salt=salt,
                     n=2**14, r=8, p=1)    # deliberately SLOW
store(email, salt, key)
# or: argon2-cffi / passlib / Django's hashers
```

Use a **slow, salted** password hash: **argon2**, **scrypt** or **bcrypt**. Compare with **hmac.compare_digest**.

TRY IT PASSWORD HASHING

Hash and verify with scrypt

```python
import hashlib, hmac, secrets

def hash_password(pw: str) -> str:
    salt = secrets.token_bytes(16)
    key = hashlib.scrypt(pw.encode(), salt=salt, n=2**14, r=8, p=1)
    return salt.hex() + "$" + key.hex()

def verify(pw: str, stored: str) -> bool:
    salt_hex, key_hex = stored.split("$")
    key = hashlib.scrypt(pw.encode(), salt=bytes.fromhex(salt_hex), n=2**14, r=8, p=1)
    return hmac.compare_digest(key.hex(), key_hex)    # constant time

a, b = hash_password("hunter2"), hash_password("hunter2")
print(a[:40] + "...")
print("same password, different hashes:", a != b)
print(verify("hunter2", a), verify("Hunter2", a))
```

OUTPUT

```
d6e79ae0863b3555a56c49b8c62d787c$56c9
b86...
same password, different hashes: True
True False
```

1. The **salt** makes every hash unique.
2. **compare_digest** avoids timing attacks.
3. **secrets** (not random) for anything security-related.

PICTURE IT JWT

What’s inside a JWT

🎫 A JWT is a **signed ticket**: anyone can **read** it, but only the server (with the secret key) can **create** a valid one. Change one character and the signature no longer matches.



Never put secrets in the payload: it’s only base64, not encrypted.

TRY IT JWT

Issue, verify, expire, tamper

```python
import base64, json, time
import jwt                                  # pip install pyjwt

SECRET = "change-me-to-a-long-random-value"
token = jwt.encode({"sub": "42", "role": "user", "exp": int(time.time()) + 900},
                   SECRET, algorithm="HS256")
print(token[:36] + "...")
print(jwt.decode(token, SECRET, algorithms=["HS256"])["role"])

head, payload, sig = token.split(".")
data = json.loads(base64.urlsafe_b64decode(payload + "=="))
data["role"] = "admin"                       # attacker edits the payload
forged = ".".join([head, base64.urlsafe_b64encode(json.dumps(data).encode()).decode().rstrip("="), sig])
try:
    jwt.decode(forged, SECRET, algorithms=["HS256"])
except jwt.InvalidSignatureError:
    print("forged token rejected")

old = jwt.encode({"sub": "42", "exp": int(time.time()) - 1}, SECRET, algorithm="HS256")
try:
    jwt.decode(old, SECRET, algorithms=["HS256"])
except jwt.ExpiredSignatureError:
    print("expired token rejected")
```

OUTPUT

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
..
user
forged token rejected
expired token rejected
```

1. Short **exp** (15 min) + a refresh token.
2. Always pass **algorithms=[...]** when decoding.
3. Logout/revocation needs extra work (a deny-list or short expiry).

STEP BY STEP SESSIONS VS TOKENS

Session cookies or JWTs?

	SESSION COOKIE	JWT (BEARER TOKEN)
State lives	on the server (DB/Redis)	in the token (stateless)
Revoke / logout	easy: delete the session	hard: wait for expiry or keep a deny-list
Best for	browser web apps (Django)	APIs, mobile apps, service-to-service
Main risk	CSRF (use SameSite + CSRF tokens)	XSS stealing a token from localStorage

For browser apps, prefer **HttpOnly** cookies; for APIs, short-lived bearer tokens. OAuth2/OIDC (Google, Entra ID) for third-party login.

STEP BY STEP OWASP

The attacks every web developer must know

ATTACK	WHAT HAPPENS	DEFENCE
SQL injection	input changes the SQL	parameterised queries / ORM
XSS	attacker's <script> runs in other users' browsers	auto-escaping templates, CSP header
CSRF	another site makes your logged-in browser POST	CSRF tokens, SameSite cookies
Broken access control	user reads /orders/7 that isn't theirs	check ownership on every request
Path traversal	../../etc/passwd	resolve the path, check it stays inside the folder
Brute force	millions of login attempts	rate limiting, lockouts, 2FA

COMMON MISTAKE ACCESS CONTROL

Checking login but not ownership

✗ WRONG

```python
@app.get("/orders/{order_id}")
def get_order(order_id: int, user=Depends(current_user)):
    return db.get(Order, order_id)
# any logged-in user can read ANY order (IDOR)
```

Being **logged in** isn't permission. Changing the number in the URL shows other customers' orders: the #1 OWASP risk.

✓ RIGHT

```python
@app.get("/orders/{order_id}")
def get_order(order_id: int, user=Depends(current_user)):
    order = db.get(Order, order_id)
    if order is None or (order.user_id != user.id
                         and not user.is_admin):
        raise HTTPException(404)          # don't reveal it exists
    return order
```

Check **ownership or role** on the server for every object.

IN REAL JOBS SECURITY

Security habits at work

Secrets never in git

```
DATABASE_URL, JWT_SECRET -> env vars / Key Vault
git-secrets / gitleaks in CI
```

Security headers

```
Strict-Transport-Security, Content-Security-Policy,
X-Content-Type-Options: nosniff
```

Dependency scanning

```
pip-audit · Dependabot · bandit
```

Login via your company IdP

```
OAuth2 / OpenID Connect (Entra ID, Google)
instead of storing passwords yourself
```

INTERVIEW SAY THIS IN THE INTERVIEW

“How do you store passwords securely?”

- Never store the password or a fast hash (MD5, SHA-256).
- Use a **slow, salted, adaptive** hash: **argon2id**, **scrypt** or **bcrypt** (frameworks have this built in).
- A unique random **salt** per user defeats rainbow tables; the work factor slows brute force.
- Verify with a constant-time comparison; rehash when you raise the cost.
- Add rate limiting, 2FA and breached-password checks.

> **Follow-up they ask next:** ”Hashing vs encryption?” (hashing is one-way, for verifying; encryption is reversible with a key, for data you need to read back).

INTERVIEW SAY THIS IN THE INTERVIEW

“How does JWT authentication work? What are its downsides?”

- On login the server issues a **signed** token with claims (sub, role, exp).
- The client sends it as **Authorization: Bearer <token>**; the server verifies the signature and expiry on every request: no session lookup.
- Downsides: hard to **revoke** before expiry, payload is **readable**, tokens in localStorage are exposed to XSS.
- Mitigate: short-lived access tokens + refresh tokens, HttpOnly cookies for browsers, rotate keys.

> **Follow-up they ask next:** ”What are XSS and CSRF and how do you prevent them?” (XSS: escape output + CSP; CSRF: CSRF tokens + SameSite cookies).

RECAP REMEMBER THESE

Module 24 in 6 lines

1. authN = who (401), authZ = what (403); check on the server.
2. Passwords: **argon2/scrypt/bcrypt** + salt; never SHA-256.
3. JWT = signed, readable, short-lived; verify signature and exp.
4. Sessions for browsers, tokens for APIs; OAuth2/OIDC for SSO.
5. OWASP: injection, XSS, CSRF, broken access control, path traversal.
6. Secrets in env/vault, security headers, rate limits, dependency scans.

PRACTICE NOW

Run 03_fastapi_login.py, then add an owner-only check to an endpoint and a test that gets 404 for someone else's order.